

DESAFIO PRÁTICO | FUNÇÕES II

Âmbito, composição, organização e testes

Tempo sugerido: 40 minutos | Sem executar código | Justifica as respostas

Nome: _____ Turma: _____ Data: _____

1 ÂMBITO DAS VARIÁVEIS

```
x = 10
def mostrar():
    x = 3
    print(x)
mostrar()
print(x)
```

- a) Escreve a saída.
- _____
- b) Explica por que existem dois valores associados ao nome x.
- _____
- _____
- _____

2 COMPOSIÇÃO DE FUNÇÕES

```
def dobro(n):
    return n * 2
def quadrado(n):
    return n * n
```

- a) Calcula quadrado(dobro(3)).
- _____
- b) Calcula dobro(quadrado(3)).
- _____
- c) Explica qual chamada termina primeiro.
- _____
- _____

3 DEVOLVER UM RESULTADO

```
def calcular():
    resultado = 5 * 2
    calcular()
    print(resultado)
```

- a) Que erro ocorre e porquê?
- _____
- b) Corrige usando return e uma variável no código principal. Não uses global.
- _____
- _____
- _____
- _____

4 FUNÇÃO PRINCIPAL E MÓDULOS

O ficheiro calculos.py contém:

```
def dobro(n):
    return n * 2
```

- a) Em programa.py, importa o módulo e mostra o dobro de 7.
- _____
- b) Para que serve a condição seguinte antes de chamar main()?
- ```
if __name__ == '__main__':
 main()
```
- \_\_\_\_\_
- \_\_\_\_\_

## 5 CORRIGIR E TESTAR

```
def media(a, b):
 return a + b / 2
```

- a) O que devolve media(10, 14)? Corrige a função para calcular a média.
- \_\_\_\_\_
- b) Escreve dois testes com assert: um para 10 e 14, outro para 0 e 0.
- \_\_\_\_\_
- \_\_\_\_\_
- \_\_\_\_\_

## 6 DESAFIO INTEGRADOR

Reorganiza o cálculo de um preço com desconto. Cria **preco\_final(preco, taxa)**, que devolve  $\text{preco} \times (1 - \text{taxa})$ , e **main()**, que lê e apresenta o resultado.

Assume preço não negativo e taxa entre 0 e 1.

a) Escreve as funções e chama main.

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

b) Testa a função com (100, 0.2) e (100, 0).

\_\_\_\_\_

# DESAFIO PRÁTICO | FUNÇÕES II

Âmbito, composição, organização e testes

## SOLUÇÕES

Compara os resultados e verifica o raciocínio.

### 1 ÂMBITO DAS VARIÁVEIS

```
3
10
```

O `x` definido dentro de mostrar é uma **variável local**. O `x` definido fora da função pertence ao âmbito global do módulo.

A atribuição local não altera o `x` global. Nomes iguais em âmbitos diferentes podem referir-se a variáveis diferentes.

### 2 COMPOSIÇÃO DE FUNÇÕES

a) `dobro(3)` devolve 6; `quadrado(6)` devolve **36**.

b) `quadrado(3)` devolve 9; `dobro(9)` devolve **18**.

c) A chamada interior termina primeiro. O valor que devolve passa a ser argumento da chamada exterior.

A ordem da composição pode alterar o resultado. `return` permite reutilizar esse resultado.

### 3 DEVOLVER UM RESULTADO

a) Ocorre **NameError**. resultado só existe no âmbito local de calcular.

```
def calcular():
 resultado = 5 * 2
 return resultado
valor = calcular()
print(valor)
```

b) A função devolve 10, que fica guardado em `valor`. A saída é **10**.

### 4 FUNÇÃO PRINCIPAL E MÓDULOS

a) Uma solução em `programa.py`:

```
import calculos
print(calculos.dobro(7))
```

A saída é **14**.

b) A condição chama `main` quando o ficheiro é executado diretamente. Ao importar esse ficheiro como módulo, esse bloco não executa.

`main` tem de estar definida antes da sua chamada.

### 5 CORRIGIR E TESTAR

a) Devolve **17.0**: primeiro calcula `14 / 2`. Faltam parênteses.

```
def media(a, b):
 return (a + b) / 2

assert media(10, 14) == 12.0
assert media(0, 0) == 0.0
```

b) Os testes passam sem produzir saída. Se uma comparação fosse falsa, `assert` lançaria `AssertionError` na execução normal.

### 6 DESAFIO INTEGRADOR

```
def preco_final(preco, taxa):
 return preco * (1 - taxa)

def main():
 p = float(input('Preço: '))
 t = float(input('Taxa: '))
 print(f'{preco_final(p, t):.2f}')

main()
```

**Testes da função:** (100, 0.2) devolve 80.0; (100, 0) devolve 100.

O cálculo fica separado da leitura e da apresentação, facilitando a reutilização e os testes.